

Lab 9: RISC Advanced I/O

Name: **Loc D. T. Nguyen**

Student ID: **24125093**

April 19, 2026

1 Exercise 3: Button-Controlled Switch Display (Prog3)

1.1 Overview

This program uses the three hardware buttons (B0, B1, B2) as commands that transform the current 10-bit switch field and display the result on the LED bank. The Button/Switch register at 0x80000010 packs 13 bits: bits [9:0] hold the switch value and bits [12:10] hold the state of B2, B1, and B0 respectively.

1.2 Algorithm and Implementation

The main loop continuously polls 0x80000010. Each iteration extracts the relevant button bit with a logical right shift followed by an `andi` mask, and branches to the appropriate handler.

Listing 1: Button Detection and Raw Display (B0 handler)

```
loop:
    lw    $11, 0($9)           # read Button/Switch register
    srl   $12, $11, 10
    andi  $13, $12, 1          # isolate B0 (bit 10)
    bne   $13, $0, b0_pressed

b0_pressed:
    andi  $14, $11, 0x3FF      # mask to 10-bit switch field
    sw    $14, 0($10)          # write to LED bank (0x8000000C)
    j     loop
```

B1 additionally left-shifts the switch value by one and re-masks to 10 bits. B2 performs bitwise inversion by XOR-ing with a mask of 0x3FF:

Listing 2: Shift and Invert Handlers (B1 and B2)

```
b1_pressed:
    andi  $14, $11, 0x3FF
    sll   $14, $14, 1
    andi  $14, $14, 0x3FF      # keep result within 10 bits
    sw    $14, 0($10)
    j     loop

b2_pressed:
    andi  $14, $11, 0x3FF
    ori   $15, $0, 0x3FF      # all-ones mask
    xor   $14, $14, $15       # flip all 10 bits
```

```
sw    $14, 0($10)
j      loop
```

1.3 Simulation Observations

- The program consists of **27 instructions**. Because all three handlers return directly to `loop`, only the most recently pressed button determines the displayed value.
- The shift-left operation requires a post-mask `andi` to prevent overflow into unused LED bits above bit 9.
- The bitwise inversion uses an intermediate register loaded with `ori $15, $0, 0x3FF` rather than an `xori` immediate, keeping the code compatible with the custom Logisim ALU.

2 Exercise 4: RAM Counter with LED Display (Prog4)

2.1 Overview

A 32-bit signed counter is maintained in data RAM at address 0x00000000. Button B0 increments it, B1 decrements it, and B2 resets it to zero. The current counter value is displayed on the 10-bit LED bank after every change.

2.2 Algorithm and Implementation

Each button handler loads the counter, modifies it with `addiu`, stores the new value back to RAM, and also writes it directly to the LED MMIO register. A release-wait loop is inserted after each action to prevent a single button press from triggering multiple increments.

Listing 3: Increment Handler with Release-Wait (B0)

```
b0_inc:
    lw    $14, 0($0)          # load counter from RAM[0x0]
    addiu $14, $14, 1
    sw    $14, 0($0)          # store updated counter
    sw    $14, 0($10)         # display on LEDs (0x8000000C)
wait_b0:
    lw    $11, 0($9)
    srl   $12, $11, 10
    andi  $13, $12, 1
    bne   $13, $0, wait_b0    # spin until B0 released
    j     loop
```

The reset handler uses `$0` directly as the store source, avoiding the need to load a zero constant:

Listing 4: Reset Handler (B2)

```
b2_reset:
    sw    $0, 0($0)          # RAM[0] = 0
    sw    $0, 0($10)         # LED = 0
```

2.3 Simulation Observations

- The program is **39 instructions**. The counter is stored as a full 32-bit signed word, so it can represent both positive and negative values as displayed on the LEDs in two's complement binary.
- Since `$0` is always zero, both the RAM store and the LED store for the reset case require no prior load, saving two instructions.
- The release-wait idiom prevents rapid repeated firing when a button is held down.

3 Exercise 5: Two-Digit 7-Segment Display Counter (Prog5)

3.1 Overview

A two-digit 7-segment display block is added to the Southbridge and mapped to address 0x80000014. The program maintains the same counter-in-RAM structure as Exercise 4, but button assignments are reversed (B0 decrements, B1 increments) and the output is directed to the 7-segment display instead of the LED bank.

3.2 Memory-Mapped I/O Extension

The new 7-segment display peripheral is wired into the Southbridge at 0x80000014. Writing a binary value v to this address causes the display controller to render the decimal representation on two digits. The MIPS processor treats it identically to any other MMIO store:

Listing 5: Initialisation and 7-Segment Store

```
ori  $16, $8, 0x0014      # $16 = 0x80000014 (7-seg address)
...
b1_inc:
    lw    $14, 0($0)
    addiu $14, $14, 1
    sw    $14, 0($0)
    sw    $14, 0($16)      # write value to 7-segment display
```

3.3 Simulation Observations

- The program is structurally identical to Prog4 (also **39 instructions**); the only differences are the button polarity (B0/B1 swapped) and the output register (\$16 replaces \$10).
- The 7-segment display decodes the binary counter value internally, so no software BCD conversion is required.
- Release-wait loops prevent repeated triggering, consistent with Exercise 4.

4 Exercise 6: 8×8 LED Matrix Arrow Symbols (Prog6)

4.1 Overview

An 8×8 LED matrix is added to the Southbridge. Its eight row registers are mapped to consecutive word addresses starting at 0x80000018 (row 0) through 0x80000034 (row 7). Each press of button B1 advances through a cycle of four arrow symbols: left ($\leftarrow$), right ($\rightarrow$), up ($\uparrow$), and down ($\downarrow$).

4.2 Symbol Design and RAM Storage

Each symbol is an 8-byte bitmap where bit 7 of each byte represents the leftmost column and bit 0 represents the rightmost. The four patterns are:

Symbol	Row 0	Row 1	Row 2	Row 3	Row 4	Row 5	Row 6	Row 7
Left ($\leftarrow$)	08	18	38	7F	7F	38	18	08
Right ($\rightarrow$)	10	18	1C	FE	FE	1C	18	10
Up ($\uparrow$)	18	3C	7E	FF	18	18	18	18
Down ($\downarrow$)	18	18	18	18	FF	7E	3C	18

The bitmaps are written into data RAM during initialisation. Symbol i , row j is stored at RAM address $4 + 32i + 4j$. The current symbol index (0–3) is kept at RAM address 0x00000000.

4.3 Algorithm and Implementation

The `display_symbol` subroutine computes the base address of the current symbol as $\$20 = \$14 \times 32 + 4$ using a left shift and immediate addition, then copies all eight rows to the LED matrix MMIO:

Listing 6: Display Subroutine

```
display_symbol:
    sll    $20, $14, 5      # $20 = index * 32
    addiu  $20, $20, 4      # $20 = base RAM address of symbol

    lw $21, 0($20) ; sw $21, 0($17)  # row 0 -> LED matrix row 0
    lw $21, 4($20) ; sw $21, 4($17)  # row 1 -> LED matrix row 1
    ...                               # rows 2-7 (same pattern)
    jr $31
```

The main loop advances the symbol index on each B1 press using a 2-bit mask to enforce modulo-4 wraparound:

Listing 7: Symbol Advance Logic

```
advance_symbol:
    lw    $14, 0($0)
    addiu $14, $14, 1
    andi  $14, $14, 3      # index mod 4 (masks to bits [1:0])
    sw    $14, 0($0)
    jal   display_symbol
```

4.4 Simulation Observations

- The program is **104 instructions**: 64 for table initialisation, 3 for setup, and the remaining 37 for the main loop, advance handler, release-wait, and subroutine.
- The modulo-4 index wrap is achieved with a single `andi $14, $14, 3` rather than a branch, exploiting the fact that $2^2 = 4$.
- The use of `jal/jr $31` keeps the display routine reusable from both the startup call and the button handler, demonstrating standard MIPS subroutine convention.

5 Conclusion

This lab explored direct hardware control on a custom MIPS Logisim processor through Memory-Mapped I/O. Exercises 3–4 established the foundational pattern of polling the combined Button/Switch register at `0x80000010`, extracting individual bit fields with shift-and-mask sequences, and reflecting computed results onto output peripherals. Exercise 5 demonstrated that adding a new I/O peripheral (the 7-segment display at `0x80000014`) requires no changes to the control logic—only the destination register changes. Exercise 6 synthesised all prior techniques: it introduced a RAM-resident data table, a `jal/jr` subroutine, and an indexed memory access pattern (`sll + addiu`) to efficiently address a multi-symbol bitmap, highlighting how a small number of MIPS instructions can drive a rich visual output with no hardware multiplier.